GOOGLE | 1POINT3ACRES | TOPIC 2 OF 9

Strings, Parsing and Recursion

11 questions to study here, each on its own page, with the full interview answer and code

5 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

1	10
HARD	MEDIUM

How to use

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- A question that fits several topics has its pages in only one PDF; the other topics list it as SKIP and name where it is.
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Strings, Parsing and Recursion | 11 questions | tap a title to jump to it

H	1. Necklace Cut into Two Halves with Equal D/R Counts	2026-05-03
M	2. Template Variable Expansion with Follow-Ups	2026-10-17
M	3. URL Request Handler Router	2026-10-16
M	4. Mahjong Winning Hand Detection (14 Tiles)	2026-10-09
M	5. Next-Word Predictor	2026-09-02
M	6. Decode String	2026-07-29
M	7. Count Word Decompositions into Chemical Element Symbols	2026-07-14
M	8. Snake Case to Camel Case with Repository-Wide Refactor	2026-07-14
M	9. Greedy Longest-Match Dictionary Tokenizer	2026-06-15
M	10. Nested Function Expression Evaluator	2026-06-10
M	11. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	2026-02-01

SKIP: these fit this topic but have their pages in another topic (5)

H	SKIP Matrix Flower Placement with House Adjacency	topic 6: Grids and Simulation
M	SKIP Nested List Weight Sum (LC 339) with Follow-Ups	topic 5: Trees and Tries
M	SKIP Target Expressions from Ordered Digits	topic 1: Arrays, Hashing and Two Pointers
M	SKIP Longest Consecutive Path in Binary Tree	topic 5: Trees and Tries
M	SKIP Fence Painting Minimum Operations (Gemini Team)	topic 3: Dynamic Programming

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Arrays, Hashing and Two Pointers, Greedy and Math (listed there as SKIP, no pages). Site tags: string, two-pointer, math-reasoning, hard

HARD | 1Point3Acres | onsite-coding | 1 of 11

Necklace Cut into Two Halves with Equal D/R Counts

ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 188/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

EXAMPLES

```
s = "DRDR"    -> cut between index 1 and 2: "DR" +
    "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR");
    rotate, two cuts at offsets 1 and 3:
              pieces "DR" and "DR".
```

APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

BRUTE FORCE TO SAY OUT LOUD

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

OPTIMAL IDEA

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

DRY-RUN THESE CASES

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

LIKELY FOLLOW-UPS

Equal strings instead of equal counts, or three pieces.

PITFALLS

INTERVIEW-DAY PLAN

Assuming a linear cut is always enough. Ignoring parity.

HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

Basis: Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate ...
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None
```

```python
# FOLLOW-UP 2, 3, 4: What if the two pieces must be equal
# strings, not just equal counts?
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


if __name__ == "__main__":
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None
```

NOTES FROM CANDIDATES

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.
Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.
Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.
Verbalize the 2-cut bound out loud before coding — interviewers ask why.

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Graphs and Search (listed there as SKIP, no pages). Site tags: string, parsing, graph, topological-sort, dfs, cycle-detection, medium

MEDIUM | 1Point3Acres | onsite-coding | 2 of 11

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 214/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

Basis: Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ...	O(depth)
exponential in the worst case	

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	O(keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict, deque


def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)
```

```
# FOLLOW-UP 1: Detect cycles before expanding.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
```

```
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Graphs and Search, Trees and Tries, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: string, parsing, data-structure, trie, dfs

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 3 of 11

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 222/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup
    for /users/123/photo/456 matches that registered
    handler.
```

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

BRUTE FORCE TO SAY OUT LOUD

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

OPTIMAL IDEA

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

LIKELY FOLLOW-UPS

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

PITFALLS

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L)	O(1)
per lookup	extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching)	O(total pattern length)
per lookup, O(total pattern length) to build	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# FOLLOW-UP 1: Different HTTP methods per route.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern


    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)


if __name__ == "__main__":
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
```

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.
The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.
Matching approach
Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.
A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.
Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.
For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization

and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: recursion, simulation, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 4 of 11

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 210/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

HOW TO THINK ABOUT IT

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import Counter


def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
def is_winning_hand(hand, melds=4):
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Arrays, Hashing and Two Pointers, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: string, hashmap, data-structure

MEDIUM | 1Point3Acres | onsite-coding | 5 of 11

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 224/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

INTERVIEW-DAY PLAN

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	O(corpus size)

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	O(number of distinct pairs)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# FOLLOW-UP 1, 2: Return the top K predictions.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None
```

```
    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.

The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.

After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.

With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.

Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Heaps, Stacks, Sorting and Binary Search (listed there as SKIP, no pages). Site tags: string, stack, parsing, medium

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 6 of 11

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 45/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

BRUTE FORCE

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
```

```
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None


if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
```

```
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

ALSO REPORTED · 1 more

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Dynamic Programming (listed there as SKIP, no pages). Site tags: string, dp, medium

MEDIUM | 1Point3Acres | phone-screen | 7 of 11

Count Word Decompositions into Chemical Element Symbols

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 81/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

EXAMPLES

Physics has four stated decompositions

P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

APPROACH

Implement a suffix-count or prefix-count formulation and state what each state represents.

Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is case-insensitive, whether to count decompositions or just test for one, and for the dictionary (the periodic table symbols of one and two letters).

BRUTE FORCE TO SAY OUT LOUD

Recursive search that tries both a one-letter and a two-letter symbol at each position. Exponential in the worst case.

OPTIMAL IDEA

Suffix-count DP: ways[i] is the number of decompositions of word[i:], with ways[n] = 1. ways[i] adds ways[i + 1] if word[i] is a symbol and ways[i + 2] if word[i:i + 2] is a symbol. O(n). Physics gives 4.

DRY-RUN THESE CASES

Physics (4), a word with no decomposition (hello), one with exactly one, the empty string, overlapping one- and two-letter symbols such as Si versus S + I.

LIKELY FOLLOW-UPS

List the decompositions, handle a larger dictionary with a trie, or count modulo a prime.

INTERVIEW-DAY PLAN

PITFALLS

Forgetting to lowercase before the lookup. Starting the DP with ways[n] = 0.

HOW TO THINK ABOUT IT

Counting the ways to cut a string into dictionary pieces is Word Break with a count instead of a yes/no. Define ways[i] as the number of decompositions of the suffix starting at i; it depends only on ways[i + 1] and ways[i + 2], because symbols have one or two letters. Compute from the end so every value you need already exists.

Basis: A counting version of LeetCode 139, Word Break, with the periodic-table symbols as the dictionary.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

TIME	SPACE
exponential (Fibonacci gr...	O(n)
exponential (Fibonacci growth)	

WHY IT IS SLOW

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION · Suffix-count DP

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

FOLLOW-UP 2 What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

FOLLOW-UP 3 List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

FOLLOW-UP 4 A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BASE QUESTION (optimal): Suffix-count DP
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]


# FOLLOW-UP 5: Count modulo a prime, or only test whether
# any decomposition exists.
def exists_decomposition(word, symbols=ELEMENTS):
    # yes / no only: the same DP with or instead of +
    return count_decompositions(word, symbols) > 0


# FOLLOW-UP 3: List the decompositions instead of counting
# them.
def list_decompositions(word, symbols=ELEMENTS):
    w, n, memo = word.lower(), len(word), {}

    def splits(i):
        # every way to split the suffix starting at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        out = []
        for size in (1, 2):
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    out.append([part] + rest)
        memo[i] = out
        return out

    return splits(0)


# FOLLOW-UP 4: A larger dictionary with long entries.
def count_with_max_len(word, symbols):
    # big dictionary: only look at pieces up to its
    # longest
    longest = max(map(len, symbols))
    w, n = word.lower(), len(word)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, min(longest, n - i) + 1):
            if w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


if __name__ == "__main__":
    assert count_decompositions("Physics") == 4
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts
    assert exists_decomposition("Physics")
    # no element starts with x then q
    assert not exists_decomposition("xq")
    assert count_decompositions("Physics", mod=3) == 1
    assert count_with_max_len("Physics", ELEMENTS) == 4
```

NOTES FROM CANDIDATES

Confirm whether matching is case-sensitive and whether the caller supplies the symbol set or the implementation may treat the periodic table as fixed.
The result counts decompositions, not merely whether at least one decomposition exists.

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Site tags: string, refactoring, medium

MEDIUM | 1Point3Acres | tech-screen | 8 of 11

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 213/228**

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

APPROACH

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HOW TO THINK ABOUT IT

Converting foo_bar to fooBar is easy. The risk is in the migration: a plain text replace also changes strings, comments, third-party names and public interfaces. Work on tokens (or the syntax tree) instead, rename only symbols defined in your own code, keep an explicit allow-list of public names, and prove nothing broke by running the tests before and after.

Basis: Written for this question; the conversion is a one-liner and the real work is a safe, token-based migration (not plain text replace).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
import ast
import io
import tokenize


# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)


# FOLLOW-UP 1: How do you prove the migration is safe?
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}


# FOLLOW-UP 1: How do you prove the migration is safe?
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

NOTES FROM CANDIDATES

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Arrays, Hashing and Two Pointers, Trees and Tries, Greedy and Math (listed there as SKIP, no pages). Site tags: string, trie, hashmap, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 9 of 11

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 215/228**

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id. Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.

Output the concatenation (or list) of ids and literal characters in order.

Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.

Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

APPROACH

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary key that is a prefix of the remaining text. Putting the keys in a trie lets you walk the text character by character from that position and remember the deepest end-of-key node you passed, instead of trying every length. If the tie-break rule changes (priority instead of length), store the priority at the terminal node and keep the best terminal seen.

Basis: Written for this question (longest-prefix matching with a trie, as in LeetCode 208 and the maximal-munch rule of lexers).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L)	O(dictionary)
lookups, each O(L)	

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	O(total key length)

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# FOLLOW-UP 1: The dictionary is huge.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
```

```
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
                break
        else:
            out.append(text[i])
            i += 1
    return out


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]
```

NOTES FROM CANDIDATES

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Heaps, Stacks, Sorting and Binary Search (listed there as SKIP, no pages). Site tags: parsing, recursion, stack, medium

MEDIUM | 1Point3Acres | onsite-coding | 10 of 11

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 175/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

EXAMPLES

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

APPROACH

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which functions exist, whether they take a fixed or variable number of arguments, whether numbers can be negative, how division behaves, and how errors are reported.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly find the innermost call with a regex and replace it with its value. Quick but fragile.

OPTIMAL IDEA

Tokenize into identifiers, integers, commas and parentheses. Write parse(i) returning (value, next index): a number returns itself; a name must be followed by '(', then loop reading arguments until ')', then apply the function. A function table keeps new operators one line each. O(n), recursion depth equals nesting depth.

DRY-RUN THESE CASES

add(1, sub(1, 0)) is 2. add(2, mul(3, pow(4, 5))) is 3074. A bare number. Variable argument counts. Unknown function names.

LIKELY FOLLOW-UPS

Variables, iterative parsing with two stacks, or division semantics.

INTERVIEW-DAY PLAN

PITFALLS

Losing the next index after parsing an argument. Treating '-' in '-5' as an operator.

HOW TO THINK ABOUT IT

An expression with function calls inside function calls has a grammar: expression = number | name '(' expression, expression, ... ')'. Each call can be evaluated only after its arguments, so write a function that parses one expression and reports where it stopped (recursive descent). A tokenizer first, then parse(i) returning (value, next index), keeps the bookkeeping small.

Basis: Recursive descent parsing, in the spirit of LeetCode 224 / 726; closest on LeetCode is 770, Basic Calculator IV.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import math
import re

TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# FOLLOW-UP 3: What about division?
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}


# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# FOLLOW-UP 2: How do you make it iterative?
def evaluate_iterative(expr):
    # two stacks: function names and their argument lists
    names, args, tokens = [], [[]], TOKEN.findall(expr)
    for tok in tokens:
        if re.fullmatch(r"-?\d+", tok):
            args[-1].append(int(tok))
        elif tok == "(":
            # the name just pushed now owns a new list
            args.append([])
        elif tok == ")":
            result = OPS[names.pop()](*args.pop())
            args[-1].append(result)
        elif tok != ",":
            names.append(tok)
    return args[0][0]


if __name__ == "__main__":
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    # variadic functions
    assert evaluate("add(1, 2, 3, 4)") == 10
    assert evaluate("max(3, min(9, 4), 1)") == 4
    # variables come from an environment
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15
    # division truncates toward zero
    assert evaluate("div(-7, 2)") == -3
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"
    assert evaluate_iterative(big) == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000
    assert evaluate_iterative(deep) == 3000
```

NOTES FROM CANDIDATES

Tokenizer + recursive descent is the cleanest approach: parse operator, consume (, parse comma-separated child expressions, consume), then evaluate.
Iterative parsing works but is easy to make off-by-one errors under interview pressure; one candidate was specifically flagged for minor syntax and bounds issues.
Clarify division behavior for negative values and non-divisible integers.

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

STUDY HERE: topic 2, Strings, Parsing and Recursion

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: string, simulation, medium

MEDIUM | 1Point3Acres | onsite-coding | 11 of 11

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 202/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space
    would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo
    bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

APPROACH

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the width counts characters in a monospace font, how to treat explicit newlines, multiple spaces and leading spaces, what to do with a word longer than the width, and whether to return the number of lines or the lines themselves.

BRUTE FORCE TO SAY OUT LOUD

Build the text line by line by trying to add each word and measuring. That is already the greedy algorithm.

OPTIMAL IDEA

Tokenize into words and newline tokens. Keep the current line length; for each word, if it fits (with a space unless the line is empty) add it, otherwise start a new line. A newline token always ends the current line. Count lines (empty lines count). Greedy is expected; mention Knuth's minimum-raggedness DP only if asked. O(n).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

The example giving 4. width = 1. Consecutive newlines. A word exactly the width. A word longer than the width.

LIKELY FOLLOW-UPS

Return the lines, justify the text, or minimize raggedness.

PITFALLS

Dropping empty lines. Adding a trailing space at the end of a line.

HOW TO THINK ABOUT IT

Greedy is optimal for the question 'fewest lines': put as many words on a line as fit. The decision per word is only 'fits with a space', 'fits as the first word on the line', or 'start a new line', so track the current line length, not the line text. Treat newline as a hard break and decide up front what empty lines and long words do.

Basis: Written; greedy word wrap (LeetCode 68, Text Justification, without the justification step). Assumed rules (confirm them): monospace width in characters; words are separated by whitespace; '\n' always ends a line and an empty line counts; a word longer than the width gets its own line.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	O(1) extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# FOLLOW-UP 2: A word longer than the width must be
# broken.
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]


# FOLLOW-UP 1: Return the lines, not only the count.
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# BASE QUESTION (optimal): Greedy with a running line
# length
def wrap_count(text, width):
    return len(wrap_lines(text, width))


# FOLLOW-UP 3: Minimise raggedness instead of lines.
def min_raggedness(words, width):
    # Knuth: minimise the squared spare space of every
    # line
    # except the last; greedy is NOT optimal for this goal
    n = len(words)
    INF = float("inf")
    # cost[i]: best total cost for words[i:]
    cost, nxt = [INF] * (n + 1), [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):
            length += len(words[j]) + 1
            if length > width:
                break
            spare = width - length
            # the last line costs nothing
            extra = 0 if j == n - 1 else spare ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                nxt[i] = j + 1
    lines, i = [], 0
    while i < n:
        lines.append(" ".join(words[i:nxt[i]]))
        i = nxt[i]
    return cost[0], lines


if __name__ == "__main__":
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == [
        "Hello", "world", "Foo bar", "baz qux"]
    assert wrap_count(text, 10) == 4
    # blank lines count as lines
    assert wrap_count("a\n\nb", 5) == 3
    # a long word is split when asked
    cut = wrap_lines("abcdefgh", 3, True)
    assert cut == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    # the best layout can beat the greedy one
    cost, lines = min_raggedness(
        "aaa bb cc ddddd".split(), 6)
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

NOTES FROM CANDIDATES

Single-pass tokenizer + accumulator: track curLen; on each token, decide "fits with leading space", "fits without space" (start of line), or "flush and start new line".
Watch out for \n right after another \n (empty line counts), \n at the end of input, leading whitespace, multiple consecutive spaces inside a line.
Don't try to handle font / kerning / hyphenation — the prompt assumes monospace width counted in characters.
Greedy packing is the expected answer (LC 68 style). Mention DP word-wrap (Knuth's algorithm) only if the interviewer pushes on 'minimum raggedness'; greedy is optimal-enough and what graders look for. Watch edge cases: a single word longer than W, an explicit newline forcing a break mid-pack, trailing whitespace handling.

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.